

MATHEMATICS REFERENCE SHEET

All Formulas & Mathematical Concepts in the CV Notebook

Companion document to: `computer-vision-notebook_v2_yolo_cascade_fixed.ipynb`

1. Activation Functions

Activation functions introduce non-linearity into neural networks. Without them, stacking layers would be equivalent to a single linear transformation — the network could never learn complex patterns.

1.1 Sigmoid

Used to convert raw logits (unbounded real numbers) into probabilities in the range (0, 1). Every binary prediction in the notebook goes through Sigmoid before a threshold is applied.

Sigmoid Function

$$\sigma(x) = 1 / (1 + e^{(-x)})$$

Output range: (0, 1)

$$\sigma(0) = 0.5 \mid \sigma(+\infty) \rightarrow 1 \mid \sigma(-\infty) \rightarrow 0$$

Notation	Description
x	Raw logit output from the neural network's final linear layer
e	Euler's number ≈ 2.71828 (base of natural logarithm)
σ(x)	Probability that the input image contains a defect

1.2 GELU (Gaussian Error Linear Unit)

Used as the activation in the classification head (replacing ReLU). GELU has smooth, non-zero gradients everywhere, which leads to more stable training. Unlike ReLU, it does not hard-zero negative inputs — it scales them by their Gaussian CDF, allowing small negative signals to flow through.

GELU Activation

$$GELU(x) = x \cdot \Phi(x)$$

where $\Phi(x) = P(X \leq x)$ for $X \sim N(0,1)$

Approximation used in practice:

$$GELU(x) \approx 0.5 \cdot x \cdot (1 + \tanh(\sqrt{2/\pi} \cdot (x + 0.044715 \cdot x^3)))$$

Notation	Description
x	Input activation value
Φ(x)	Cumulative distribution function (CDF) of the standard normal distribution N(0,1)

tanh	Hyperbolic tangent: $\tanh(x) = (e^x - e^{-x}) / (e^x + e^{-x})$
-------------	--

2. Loss Functions

Loss functions measure how wrong the model's predictions are. Training minimises the loss via gradient descent. Three loss functions are combined in this notebook.

2.1 Binary Cross-Entropy (BCE)

The foundation of all three loss functions.

Binary Cross-Entropy Loss

$$BCE(\hat{y}, y) = -[y \cdot \log(\hat{y}) + (1-y) \cdot \log(1-\hat{y})]$$

With pos_weight w (class imbalance correction):

$$BCE_w(\hat{y}, y) = -[w \cdot y \cdot \log(\hat{y}) + (1-y) \cdot \log(1-\hat{y})]$$

Averaged over a batch of N samples:

$$L_{BCE} = (1/N) \cdot \sum_i BCE(\hat{y}_i, y_i)$$

Notation	Description
$y \in \{0, 1\}$	True label (0 = good rim, 1 = faulty rim)
$\hat{y} = \sigma(z)$	Predicted probability after Sigmoid applied to logit z
w	pos_weight: ratio of negative samples to positive samples, clamped to [1, 10]. Penalises misclassified faulty rims more heavily.
log	Natural logarithm (base e)
N	Batch size

2.2 Focal Loss

Extends BCE to down-weight easy, well-classified examples. On a dataset where 90% of rims are good, standard BCE is dominated by easy negatives (correct 'good' predictions). Focal Loss multiplies each sample's BCE by a factor that approaches 0 for confidently-correct predictions and stays near 1 for hard, ambiguous ones.

Focal Loss

$$FL(\hat{y}, y) = -(1 - p_t)^\gamma \cdot [w \cdot y \cdot \log(\hat{y}) + (1-y) \cdot \log(1-\hat{y})]$$

where $p_t = \hat{y} \cdot y + (1-\hat{y}) \cdot (1-y)$

With label smoothing ($\epsilon = 0.02$):

$$y_{smooth} = y \cdot (1 - \epsilon) + 0.5 \cdot \epsilon$$

$\gamma = 2.0$ (focusing parameter used in notebook)

Notation	Description
p_t	Probability assigned to the correct class. High $p_t \rightarrow$ easy sample $\rightarrow$ low loss weight.
$(1 - p_t)^\gamma$	Modulating factor. When $p_t = 0.9$ (easy), factor = 0.01. When $p_t = 0.1$ (hard), factor = 0.81.
γ (gamma)	Focusing parameter. $\gamma=0 \rightarrow$ standard BCE. $\gamma=2 \rightarrow$ strong focus on hard examples.
ϵ (epsilon)	Label smoothing factor = 0.02. Softens hard labels to prevent overconfidence.

2.3 Asymmetric Loss (ASL)

Uses different focusing exponents for positive (defective) and negative (good) samples. This asymmetry allows more aggressive suppression of easy negatives while maintaining sensitivity to rare positives.

Asymmetric Loss (as implemented in notebook)

$$ASL(\hat{y}, y) = -[\gamma \cdot (1 - \hat{y})^{\gamma^+} \cdot \log(\hat{y}) + (1 - \gamma) \cdot \hat{y}^{\gamma^-} \cdot \log(1 - \hat{y}_m)]$$

where $\hat{y}_m = \min(\hat{y} + \delta, 1)$ (probability shift / clip)

$\gamma^+ = 1$ (positive focusing, notebook default)

$\gamma^- = 4$ (negative focusing, notebook default)

$\delta = 0.05$ (clip margin)

Note: The implemented ASL differs from the original paper (Ridnik et al.) in two ways: (1) the clip is added ($\hat{y} + \delta$) rather than subtracted, and (2) the focusing weight uses the raw probability $\hat{y}^{\gamma^-}$ rather than the shifted value $\hat{y}_m^{\gamma^-}$.

Notation	Description
γ^+ (gamma_pos)	Focusing exponent for positive (faulty) samples. Lower = less focusing.
γ^- (gamma_neg)	Focusing exponent for negative (good) samples. Higher = more aggressive suppression of easy negatives.
$\hat{y}_m$	Probability after applying the additive shift δ . Easy negative samples ($\hat{y}$ close to 0) are

	shifted to $\sim\delta$, contributing reduced gradient.
δ (clip)	Margin = 0.05. Added to negative predictions before computing $\log(1 - \hat{y}_m)$, reducing the penalty on easy negatives.

2.4 Combined Loss

The actual training objective is a weighted combination:

Combined Loss (final objective)

$$L_{total} = (1 - \alpha) \cdot FL(\hat{y}_{bin}, y) + \alpha \cdot ASL(\hat{y}_{bin}, y) + \lambda \cdot MSE(\sigma(z_{sev}), y)$$

$\alpha = 0.40$ (ASL weight)

$\lambda = 0.25$ (severity head weight)

$$MSE(a, b) = (1/N) \cdot \sum_i (a_i - b_i)^2$$

Notation	Description
FL	Focal Loss on the binary classification head logits
ASL	Asymmetric Loss on the same binary logits
α	ASL blend weight = 0.40. 40% of binary loss comes from ASL, 60% from Focal.
z_{sev}	Output of the secondary severity head (a separate linear layer)
$\sigma(z_{sev})$	Sigmoid of severity logit, treated as a soft defect severity score
MSE	Mean Squared Error — penalises the severity head for being far from the binary label
λ	Severity loss weight = 0.25. Acts as a regulariser, not a primary objective.

3. Attention Mechanisms

Attention mechanisms allow the network to dynamically weight different parts of its input — focusing on defect-relevant features and suppressing background noise.

3.1 Channel Attention (CBAM — Channel Branch)

Determines which feature channels are useful for defect detection.

CBAM Channel Attention

$$M_c(F) = \sigma(MLP(AvgPool(F)) + MLP(MaxPool(F)))$$

$$MLP(v) = W_2 \cdot ReLU(W_1 \cdot v)$$

$$\tilde{F} = M_c(F) \otimes F$$

Notation	Description
$F \in \mathbb{R}^{(C \times H \times W)}$	Input feature map: C channels, H height, W width
$AvgPool(F)$	Global average pooling → vector of shape (C,). Mean activation per channel.
$MaxPool(F)$	Global max pooling → vector of shape (C,). Peak activation per channel.
MLP	Shared 2-layer network: W_1 compresses $C \rightarrow C/r$ ($r=16$), W_2 expands back $C/r \rightarrow C$
σ	Sigmoid — outputs a weight in (0,1) per channel
$M_c(F) \in (0,1)^C$	Channel attention weights — one scalar per channel
$\otimes$	Element-wise multiplication (broadcast across spatial $H \times W$ dimensions)
$\tilde{F}$	Channel-recalibrated feature map

3.2 Spatial Attention (CBAM — Spatial Branch)

Determines where in the image to focus.

CBAM Spatial Attention

$$M_s(\tilde{F}) = \sigma(f^{(7 \times 7)}([AvgPool_c(\tilde{F}) ; MaxPool_c(\tilde{F})]))$$

$$F_{out} = M_s(\tilde{F}) \otimes \tilde{F}$$

Notation	Description
$AvgPool_c(\tilde{F})$	Channel-wise average pooling → $1 \times H \times W$ map (mean across all C channels at each pixel)
$MaxPool_c(\tilde{F})$	Channel-wise max pooling → $1 \times H \times W$ map (peak across all C channels at each pixel)
$[;]$	Concatenation along channel dimension → $2 \times H \times W$ tensor
$f^{(7 \times 7)}$	Convolution with a 7×7 kernel, padding=3 to preserve $H \times W$
$M_s(\tilde{F}) \in (0,1)^{(H \times W)}$	Spatial attention map — one weight per pixel location
F_{out}	Final CBAM output: spatially and channel-recalibrated feature map

3.3 Attention Pooling (AttentionPool2d)

Replaces Global Average Pooling at the end of each backbone.

Attention Pooling

$$s(x) = \text{Conv}_{\{1 \times 1\}}(x) \in \mathbb{R}^{(1 \times H \times W)}$$

$$w = \text{Softmax}(\text{reshape}(s(x), [B, H \cdot W])) \in \mathbb{R}^{(B \times H \cdot W)}$$

$$z = \sum_{\{h, w\}} w_{\{h, w\}} \cdot x_{\{:, h, w\}} \in \mathbb{R}^{(B \times C)}$$

Notation	Description
$x \in \mathbb{R}^{(B \times C \times H \times W)}$	Feature map from backbone: B=batch, C=channels, H×W=spatial dims
$\text{Conv}_{\{1 \times 1\}}$	1×1 convolution — learns a single scalar score per spatial location
Softmax	Normalises scores across all H·W positions to sum to 1
$w_{\{h, w\}}$	Attention weight assigned to spatial location (h, w)
z	Weighted sum of feature vectors — a defect-focused global descriptor

3.4 Multi-Head Cross-Attention (Backbone Fusion)

Combines the three backbone outputs via Transformer-style attention.

Scaled Dot-Product Attention

$$\text{Attention}(Q, K, V) = \text{Softmax}(Q \cdot K^T / \sqrt{d_k}) \cdot V$$

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) \cdot W^o$$

Input: tokens = [f_eff, f_rn, f_cx] $\in \mathbb{R}^{(B \times 3 \times 320)}$

h = 8 heads, d_k = 320/8 = 40

Notation	Description
Q, K, V	Query, Key, Value matrices — all equal to the token sequence (self-attention)
d_k = 40	Dimensionality per head. Dividing by $\sqrt{d_k}$ prevents dot-products from growing too large.
h = 8	Number of attention heads. Each head learns a different relationship between backbones.
W_i^k, W_i^v, W^o	Learnable projection matrices for keys, values, and output

f_eff, f_rn, f_cx	320-d projected features from EfficientNet, RegNet, and ConvNeXt respectively
--------------------------	---

4. Normalisation

4.1 ImageNet Normalisation

Per-Channel Z-Score Normalisation

$$x'_c = (x_c - \mu_c) / \sigma_c$$

$\mu = [0.485, 0.456, 0.406]$ (ImageNet mean, RGB channels)

$\sigma = [0.229, 0.224, 0.225]$ (ImageNet std, RGB channels)

Notation	Description
x_c ∈ [0, 1]	Raw pixel value in channel c (after dividing uint8 by 255)
μ_c	Mean pixel value of channel c across the entire ImageNet training set
σ_c	Standard deviation of channel c across the entire ImageNet training set
x'_c	Normalised pixel value. Roughly in range [−2.5, 2.5] for natural images.

4.2 Batch Normalisation (BatchNorm)

Batch Normalisation

$$\mu_B = (1/m) \cdot \sum_i x_i$$

$$\sigma^2_B = (1/m) \cdot \sum_i (x_i - \mu_B)^2$$

$$\hat{x}_i = (x_i - \mu_B) / \sqrt{\sigma^2_B + \epsilon}$$

$$y_i = \gamma \cdot \hat{x}_i + \beta$$

Notation	Description
m	Number of samples in the current mini-batch
μ_B, σ²_B	Mean and variance computed over the current batch
ε = 1e-5	Small constant added for numerical stability
γ, β	Learnable scale and shift parameters (one pair per feature channel)
ŷ_i	Normalised activation — approximately zero mean, unit variance

4.3 Layer Normalisation (LayerNorm)

Used in the Cross-Attention fusion block.

Layer Normalisation

$$\mu = (1/d) \cdot \sum_j x_j$$

$$\sigma^2 = (1/d) \cdot \sum_j (x_j - \mu)^2$$

$$LN(x)_j = \gamma_j \cdot (x_j - \mu) / \sqrt{\sigma^2 + \epsilon} + \beta_j$$

Notation	Description
d = 320	Feature dimension (normalise across all 320 values per token)
γ_j, β_j	Per-feature learnable scale and shift

5. Regularisation Techniques

5.1 Dropout

Dropout

During training:

$$mask \sim \text{Bernoulli}(1-p)$$

$$y = (x \odot mask) / (1-p) \quad (\text{inverted dropout})$$

During inference:

$$y = x$$

p = 0.35 (uniform across all classification head layers)

Notation	Description
p	Dropout rate. p=0.35 means 35% of neurons are zeroed each forward pass.
Bernoulli(q)	Random variable = 1 with probability q, 0 with probability 1-q
$\odot$	Element-wise (Hadamard) multiplication
$/(1-p)$	Inverted scaling: keeps the expected sum of outputs constant between train and inference

5.2 MixUp Augmentation

MixUp

$$\lambda \sim \text{Beta}(\alpha, \alpha)$$

$$\tilde{x} = \lambda \cdot x_i + (1-\lambda) \cdot x_j$$

$$\tilde{y} = \lambda \cdot y_i + (1-\lambda) \cdot y_j$$

$\alpha = 0.05$ (notebook default)

Notation	Description
$\lambda \in (0,1)$	Random blend coefficient sampled from Beta distribution
$\text{Beta}(\alpha,\alpha)$	Symmetric Beta distribution. At $\alpha=0.05$ it is sharply U-shaped: λ is usually near 0 or 1.
x_i, x_j	Two randomly paired images from the same batch
$y_i, y_j \in \{0,1\}$	Their corresponding binary labels
$\tilde{x}, \tilde{y}$	Mixed image and soft label passed to the loss function

5.3 CutMix Augmentation

CutMix

Box size: $\text{cut_r} = \sqrt{(1 - \lambda_\beta)}$

$\text{cut_h} = H \cdot \text{cut_r}$, $\text{cut_w} = W \cdot \text{cut_r}$

Paste:

$\tilde{x}[:, y1:y2, x1:x2] = x_j[:, y1:y2, x1:x2]$

Actual area ratio:

$$\lambda_{\text{actual}} = 1 - (x2-x1) \cdot (y2-y1) / (H \cdot W)$$

Mixed label:

$$\tilde{y} = \lambda_{\text{actual}} \cdot y_i + (1-\lambda_{\text{actual}}) \cdot y_j$$

Notation	Description
$\lambda_\beta \sim \text{Beta}(\alpha,\alpha)$	Initial beta sample determining box size (renamed from 'lam' per bug fix)
cut_r	Box side length as a fraction of image dimension
$(x1,y1), (x2,y2)$	Bounding box corners, clipped to image

	boundaries
λ_{actual}	True proportion of original image remaining after paste. Used for label blending.

5.4 L2 / Weight Decay

L2 Regularisation (Weight Decay)

$$L_{\text{total}} = L_{\text{task}} + (\lambda_{\text{wd}} / 2) \cdot ||\vartheta||^2$$

Gradient update with weight decay:

$$\vartheta \leftarrow \vartheta \cdot (1 - \eta \cdot \lambda_{\text{wd}}) - \eta \cdot \nabla_{\vartheta} L_{\text{task}}$$

Phase 1 & 2: $\lambda_{\text{wd}} = 1\text{e-}4$

Phase 3: $\lambda_{\text{wd}} = 5\text{e-}5$

Notation	Description
$ \vartheta ^2$	L2 norm squared of all trainable parameters: $\sum \theta_i^2$
λ_{wd}	Weight decay coefficient. Smaller in Phase 3 for fine-grained adaptation.
η	Learning rate

6. Optimiser & Learning Rate Schedules

6.1 AdamW Optimiser

AdamW Update Rule

$$m_t = \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot g_t$$

$$v_t = \beta_2 \cdot v_{t-1} + (1 - \beta_2) \cdot g_t^2$$

$$\hat{m}_t = m_t / (1 - \beta_1^t) \quad (\text{bias correction})$$

$$\hat{v}_t = v_t / (1 - \beta_2^t) \quad (\text{bias correction})$$

$$\vartheta_t = \vartheta_{t-1} - \eta \cdot (\hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon) + \lambda_{\text{wd}} \cdot \vartheta_{t-1})$$

$\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 1\text{e-}8$ (PyTorch defaults)

Notation	Description
g_t	Gradient of loss w.r.t. parameter θ at step t
m_t	Exponentially weighted moving average of

	gradients (momentum)
$\mathbf{v}_t$	Exponentially weighted moving average of squared gradients (adaptive scale)
$\beta_1 = 0.9$	Decay rate for 1st moment
$\beta_2 = 0.999$	Decay rate for 2nd moment
$1/(\sqrt{\hat{\mathbf{v}}_t} + \epsilon)$	Adaptive step size
$\lambda_{wd} \cdot \theta_{t-1}$	Decoupled weight decay

6.2 Cosine Annealing with Warm Restarts

Cosine Annealing LR Schedule

$$\eta_t = \eta_{min} + \frac{1}{2} \cdot (\eta_{max} - \eta_{min}) \cdot (1 + \cos(\pi \cdot t / T_o))$$

$T_o = 7$ epochs (restart period)

$\eta_{min} = 1e-7$, $\eta_{max} = \text{base_lr} = 5e-5$

6.3 Linear Warmup (Phase 3 Only)

Linear Warmup

$$\eta_t = \eta_{start} + (\eta_{end} - \eta_{start}) \cdot (t / T_{warmup})$$

$\eta_{start} = 0.1 \cdot \text{base_lr}$, $\eta_{end} = \text{base_lr}$

$T_{warmup} = \text{len}(\text{train_loader}) / \text{accum_steps} (\approx 1 \text{ epoch})$

6.4 SWA Cosine Annealing (SWALR)

SWALR

$$\eta_{swa}(e) = \text{swa_lr} + \frac{1}{2} \cdot (\eta_{start} - \text{swa_lr}) \cdot (1 + \cos(\pi \cdot e / T_{anneal}))$$

$\text{swa_lr} = 5e-6$, $T_{anneal} = 5$ epochs

7. Model Averaging

7.1 Exponential Moving Average (EMA)

EMA Weight Update

$$d(t) = \min(\delta, (1 + t) / (10 + t)) \quad (\text{warm-up decay})$$

$$\vartheta_{shadow} \leftarrow d(t) \cdot \vartheta_{shadow} + (1 - d(t)) \cdot \vartheta_{model}$$

$\delta = 0.9998$ (target decay, notebook default)

Effective half-life $\approx \log(0.5) / \log(0.9998) \approx 3465$ steps

Notation	Description
θ_{shadow}	EMA shadow model weights — never receives direct gradients
θ_{model}	Main model weights updated by the optimiser
$d(t)$	Effective decay at step t. Starts near 0 (fast update) and converges to δ (slow update).
$(1+t)/(10+t)$	Warm-up formula: at $t=0$ gives $1/10=0.1$; as $t \rightarrow \infty$ gives 1.0
$\delta = 0.9998$	At full decay, the shadow retains 99.98% of old weights and absorbs 0.02% of new.

7.2 Stochastic Weight Averaging (SWA)

SWA Model Average

$$\vartheta_{\text{SWA}}^{(n+1)} = (n \cdot \vartheta_{\text{SWA}}^{(n)} + \vartheta_{\text{model}}) / (n + 1)$$

n = number of models averaged so far (increments each epoch)

BatchNorm recalibration is required after SWA: BN running stats are stale because ϑ_{SWA} was never trained end-to-end. 100 batches of real data recalculate μ_B and σ^2_B .

8. Evaluation Metrics

8.1 Confusion Matrix Quantities

Confusion Matrix

$$TP = |\{i : y_i=1 \wedge \hat{y}_i=1\}| \quad (\text{correct faulty detections})$$

$$TN = |\{i : y_i=0 \wedge \hat{y}_i=0\}| \quad (\text{correct good detections})$$

$$FP = |\{i : y_i=0 \wedge \hat{y}_i=1\}| \quad (\text{false alarms})$$

$$FN = |\{i : y_i=1 \wedge \hat{y}_i=0\}| \quad (\text{missed defects})$$

$$N = TP + TN + FP + FN \quad (\text{total samples})$$

8.2 Precision, Recall, Accuracy, F1

Core Classification Metrics

$$\text{Accuracy} = (TP + TN) / N$$

$$\text{Precision} = TP / (TP + FP)$$

$$\text{Recall (Sensitivity)} = TP / (TP + FN)$$

$$F1 = 2 \cdot \text{Precision} \cdot \text{Recall} / (\text{Precision} + \text{Recall}) = 2 \cdot TP / (2 \cdot TP + FP + FN)$$

Notation	Description
Precision	Of all predicted positives, what fraction are truly positive? High precision = few false alarms.
Recall	Of all true positives, what fraction were predicted positive? High recall = few missed defects.
F1	Harmonic mean of Precision and Recall. Primary competition metric.

8.3 AUC-ROC

ROC Curve & AUC

$$TPR(\tau) = TP(\tau) / (TP(\tau) + FN(\tau)) \quad (\text{True Positive Rate} = \text{Recall})$$

$$FPR(\tau) = FP(\tau) / (FP(\tau) + TN(\tau)) \quad (\text{False Positive Rate})$$

ROC curve: plot $TPR(\tau)$ vs. $FPR(\tau)$ as τ sweeps from $1 \rightarrow 0$

AUC = 0.5 $\rightarrow$ random classifier

AUC = 1.0 $\rightarrow$ perfect classifier

8.4 Threshold Optimisation

Vectorised Threshold Sweep

$$\tau \in \{0.05, 0.06, \dots, 0.95\} \quad (91 \text{ thresholds})$$

$$\hat{y}_i(\tau) = 1[\sigma(z_i) \geq \tau] \quad (\text{binary prediction at threshold } \tau)$$

$$F1(\tau) = 2 \cdot TP(\tau) / (2 \cdot TP(\tau) + FP(\tau) + FN(\tau))$$

$$\tau^* = \operatorname{argmax}_{\tau} F1(\tau)$$

9. Probability Calibration (Platt Scaling)

Platt Scaling (Logistic Regression Calibration)

$$\tilde{P}(y=1 | z) = 1 / (1 + \exp(-(A \cdot z + B)))$$

A, B found by minimising NLL on validation set:

$$(A^*, B^*) = \operatorname{argmin}_{\{A, B\}} \sum_i -y_i \cdot \log(\tilde{P}_i) - (1 - y_i) \cdot \log(1 - \tilde{P}_i)$$

Implemented as: `LogisticRegression(C=1.0).fit(z_val, y_val)`

Notation	Description
$z = \sigma(\text{logit})$	Raw sigmoid probability from the neural network (input to calibrator)
A, B	Learnable scalar parameters of the calibration curve
$\tilde{P}$	Calibrated probability. If $\tilde{P}=0.8$, approximately 80% of such cases are truly faulty.
NLL	Negative Log-Likelihood — same as Binary Cross-Entropy
C = 1.0	Inverse regularisation strength in sklearn's LogisticRegression

10. Computer Vision Mathematics

10.1 2D Convolution

Discrete 2D Convolution

$$(F * K)[i,j] = \sum_m \sum_n F[i+m, j+n] \cdot K[m, n]$$

Output spatial size:

$$H_{\text{out}} = \lfloor (H_{\text{in}} + 2 \cdot \text{pad} - \text{kernel}) / \text{stride} \rfloor + 1$$

Notation	Description
$F[i,j]$	Input feature map value at row i, column j
$K[m,n]$	Convolution kernel (filter) of size k×k — learnable weights
$*$	Convolution operator
pad	Zero-padding added to input borders to control output size
stride	Step size of the filter across the input

10.2 Global Average Pooling (GAP) vs. Attention Pooling

Global Average Pooling

$$\text{GAP}(F)_c = (1 / H \cdot W) \cdot \sum_h \sum_w F_c[h, w]$$

Problem: uniform weights → 1% defect area diluted to 1% signal

Attention Pooling (as used in notebook)

$$AttPool(F)_c = \sum_h \sum_w softmax(s[h,w]) \cdot F_c[h,w]$$

$s[h,w] = Conv_{\{1 \times 1\}}(F)[h,w]$ (learned scalar score per location)

10.3 Hough Circle Transform

Used in the inference pipeline to locate the bottle rim before cropping.

Hough Circle Transform

$$(x - a)^2 + (y - b)^2 = r^2$$

For each edge pixel (x_0, y_0) and candidate radius r , cast votes to all (a, b) satisfying the equation.

Peak in accumulator space $(a^*, b^*, r^*) \rightarrow$ detected circle

Gradient direction shortcut: $a = x_0 - r \cdot \cos(\theta)$, $b = y_0 - r \cdot \sin(\theta)$

10.4 Gaussian Blur (Pre-processing for HoughCircles)

2D Gaussian Kernel

$$G(x,y) = (1 / 2\pi\sigma^2) \cdot \exp(-(x^2 + y^2) / 2\sigma^2)$$

Applied with kernel (9×9) , $\sigma=2$ to suppress noise before HoughCircles.

10.5 Specular Highlight (2D Gaussian Mask)

Specular Highlight Gaussian Mask

$$mask(x,y) = \exp(-((x-c_x)/r_x)^2 + ((y-c_y)/r_y)^2)$$

$$I'(x,y,c) = I(x,y,c) + \alpha \cdot mask(x,y) \cdot 255$$

$\alpha \sim \text{Uniform}(0.2, 0.7)$, (c_x, c_y) random centre, (r_x, r_y) random radii

Notation	Description
$mask(x,y) \in [0,1]$	Elliptical Gaussian weight — 1 at centre, decaying to 0 at edges
(c_x, c_y)	Random centre of the highlight, within the central half of the image
(r_x, r_y)	Random semi-axes of the ellipse controlling highlight spread
α	Intensity multiplier — how bright the glare is

11. Test-Time Augmentation — D4 Symmetry Group

TTA averages predictions across 8 transformed views. The 8 views correspond to the D4 dihedral group — all symmetries of the square.

D4 Dihedral Group (8 unique transforms)

$$D4 = \{ e, r, r^2, r^3, s, sr, sr^2, sr^3 \}$$

- e = identity (no transform)
- r = rotate 90° (counterclockwise)
- r^2 = rotate 180°
- r^3 = rotate 270°
- s = flip horizontal
- sr = flip horizontal + rotate 90°
- sr^2 = flip vertical (= flip horizontal + rotate 180°)
- sr^3 = flip vertical + rotate 90°

TTA Prediction

$$P_{TTA}(y=1 \mid x) = (1/8) \cdot \sum_{g \in D4} \sigma(f_{\vartheta}(g(x)))$$

Notation	Description
D4	Dihedral group of order 8 — the 8 symmetries of a square
$g \in D4$	One of the 8 unique image transformations
$g(x)$	Transformed image: the same rim seen from a different orientation
$(1/8) \cdot \Sigma$	Simple mean of the 8 sigmoid probabilities — reduces variance

Why D4? A square image has exactly 8 distinct orientations. A 16-view approach (4 flips $\times$ 4 rotations) generates duplicates, biasing the average.

End of Mathematics Reference Sheet